

GeoWM: Efficient Direct World Modeling in Explicit Geometry

Mehrdad Noori, Guile Wu, Sam Hosseini, and Dongfeng Bai
Huawei Noah’s Ark Lab, Canada

Abstract—Modeling 3D scene geometry and its evolution over time is essential for autonomous driving and robotics. A common paradigm is to use world models to predict future images or latent representations of the environment and subsequently recover geometry from these predictions. However, this paradigm does not explicitly model geometric structure and typically relies on recursive rollouts to reach longer prediction horizons, leading to error accumulation and increasing computational cost. To address these limitations, we present GEOWM, a geometry world model that directly forecasts future scene geometry at specified future horizons without recursive rollout. The key idea is to leverage a geometry foundation model to transform observed RGB frames into a geometric history, which conditions a flow-matching transformer to predict the scene geometry at a specified future horizon. We further show that a lightweight camera-motion predictor can accurately estimate the future viewpoint, and that projecting the observed geometry into the predicted viewpoint provides an effective geometric prior for future geometry forecasting. Extensive experiments on four datasets spanning urban driving, aerial flight, and dynamic manipulation demonstrate that GEOWM outperforms the evaluated world models in forecasting depth, camera pose, and 3D scene geometry, while substantially reducing inference time at longer horizons.

I. INTRODUCTION

Understanding the 3D structure of the environment and how it evolves over time is crucial for tasks such as autonomous driving and robotic navigation. Recent advances in geometry foundation models [1]–[4] have shown impressive capabilities in modeling 3D scene geometry and camera parameters from images. However, these models are inherently observational as they focus on reconstructing the present scene within the given images rather than predicting its future evolution. As a result, they lack the ability to forecast how the scene will evolve as the camera and objects move, which is crucial for planning and control.

On the other hand, world models have shown promise in predicting future states of the environment as images or latent representations, from which scene geometry can subsequently be inferred. DINO-Foresight [5] forecasts the semantic features of a frozen vision foundation model and attaches task heads to read depth from them. Video generators such as Cosmos [6] synthesise future RGB frames, from which geometry is reconstructed afterwards. VGGT-World [7] moves closer to geometry, forecasting the feature states of a frozen geometry foundation model and decoding them with its pretrained heads. Although these approaches demonstrate the potential of world models for geometry forecasting, they struggle with representing complex 3D geometry accurately (Fig. 1). Moreover, existing methods often employ an autoregressive rollout strategy, feeding each

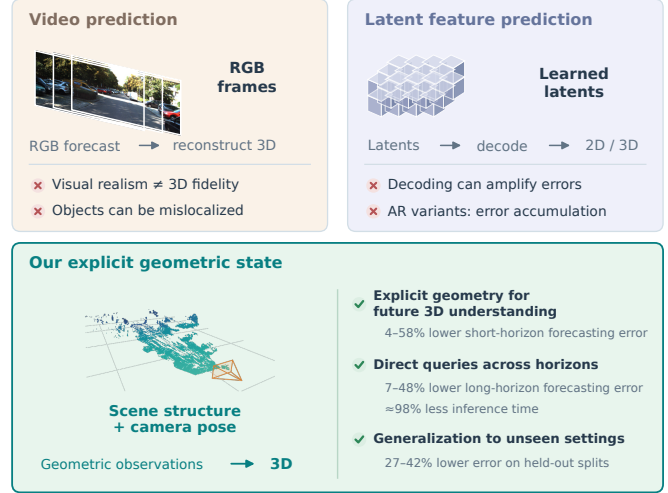


Fig. 1: **Pixels, features, or explicit geometry?** Video models forecast future frames and then reconstruct geometry; their photometric objective couples only loosely to 3D structure, and dynamic objects can be mislocalised [7]. Feature-space models forecast latents and decode them, where decoding can amplify errors [7] and autoregressive rollout accumulates them [8]–[10]. GEOWM forecasts explicit geometry directly and queries each horizon independently.

prediction back as the input to the next, which makes maintaining long-term consistency over extended horizons challenging as errors accumulate over time and computational cost increases.

In this work, we propose GEOWM, a geometry world model that combines explicit geometry modeling with direct future prediction at specified future horizons. Specifically, we employ a geometry foundation model to extract geometric information from observed frames, forming a geometric history that serves as the condition for future geometry prediction. We then use a simple yet effective flow-matching diffusion transformer to predict the future geometry based on the observed geometric history, the predicted camera motion, and the requested future horizon. Each requested future horizon is predicted directly, without the need for recursive rollouts, which mitigates error accumulation and reduces computational cost. Furthermore, we employ a lightweight camera-motion predictor to estimate the future viewpoint, which is then used to project the observed geometry into the predicted viewpoint, providing an effective geometric prior for future geometry forecasting. In this way, GEOWM integrates explicit geometry modeling, direct future prediction,

and camera-motion estimation into a cohesive framework for accurate and efficient 3D scene forecasting.

We evaluate GEOWM on four datasets, ranging from urban driving and aerial flight to robotic manipulation, at different horizons. We compare against three recent state-of-the-art world models: the video-based Cosmos-3 [6], the latent-geometry VGGT-World [7], and the feature-based DINO-Foresight [5]. Across three families of metrics, future depth, future camera pose, and future 3D point clouds, GEOWM outperforms other methods across horizons. It lowers depth error by 4–58% at short horizons and 7–48% at long horizons against the strongest comparable baseline, with about 98% less inference time. Its predicted 3D point clouds are also the most accurate on all four datasets. Together with controlled ablations of each design choice, these results show that forecasting directly in explicit geometry is both more accurate and more efficient than forecasting in pixel or feature space, and they offer a practical route to world models for the many robotic tasks that depend on 3D understanding.

In summary, our contributions are:

- We introduce a geometry world model that forecasts future 3D scene geometry in an explicit geometry space extracted from observed RGB frames, providing a structured and interpretable representation for future prediction.
- We develop a horizon-conditioned diffusion transformer that predicts the requested future geometry directly without recursive rollouts, mitigating error accumulation, so the cost of a forecast does not grow with the horizon.
- Our method is efficient and outperforms previous state-of-the-art world models in future geometry forecasting at different horizons on driving, aerial, and robotic manipulation datasets, demonstrating the potential of explicit geometry as the state of a world model for tasks that depend on 3D understanding.

II. RELATED WORK

Geometry Foundation Models. Feed-forward reconstruction offers an efficient alternative to optimisation-based pipelines for obtaining 3D structure from images. DUST3R [1] showed that a single network can regress point maps, and hence depth and relative camera pose, from unposed image pairs, and VGGT [2] extended this to many views with a transformer that predicts camera parameters, depth maps, and point maps in one forward pass. Subsequent models improve the formulation, for example by removing the dependence on a reference view [3] or by accepting metric priors and calibration as optional inputs [4]. These models describe the frames they are given with high accuracy, and we use one of them, frozen, to turn observed RGB frames into geometry. They are not designed to predict how that geometry will change; providing this is the goal of the present work.

Video World Models. Large video generators such as Cosmos [6], [11] and Wan [12], and driving-specific models such as GAIA-1 [13] and Vista [14], are used as world models

by rolling out future frames, from which geometry can then be reconstructed. Geometry-aware variants generate depth or point maps jointly with the frames [15]–[17] or align the generator’s features with those of a geometry model [18]. In both cases the forecaster remains a video model whose capacity is spent on appearance, and geometry is obtained either after generation or as one of its outputs. Many video models generate bounded clips and extend them through autoregressive feedback, and the mismatch this creates between training on real frames and generating from predicted ones is treated as a training problem: scheduled sampling [8] and Self Forcing [10] expose the model to its own generated context during training, and Diffusion Forcing [9] trains on noise-corrupted context so that imperfect history is seen at training time. These remedies reduce drift but keep the sequential mechanism and its cost, which grows with the horizon.

Latent World Models. A second family forecasts in learned visual feature spaces. DINO-Foresight [5] and DINO-World [19] predict the future features of a frozen DINOv2 encoder and read out depth or segmentation with separately trained task heads, and V-JEPA 2 [20] trains encoder and predictor jointly for action anticipation and planning. DINO-World conditions on absolute timestamps, so a future time can be queried directly as well as reached autoregressively. Most recently, VGGT-World [7] moved the latent space to a geometry foundation model: it predicts the tokens of a frozen VGGT and decodes them with the backbone’s own pretrained heads, so the forecast is tied directly to camera parameters, depth maps, and point maps. This is cheaper and closer to geometry than pixel space, but the geometry is still decoded from the forecast rather than being the forecast. Latent models also roll out in the same way as video models: DINO-Foresight predicts features one step at a time, and VGGT-World predicts chunks of tokens, feeds them back, and relies on a flow-forcing curriculum to limit the resulting drift. Compared with VGGT-World, our method forecasts explicit geometric quantities and predicts each requested horizon directly, without recursive feedback, using predicted camera motion to condition the geometry forecast.

III. METHOD

GEOWM forecasts future scene geometry directly in the output space of a geometry foundation model (Fig. 2). Given the K most recent RGB frames, a frozen reconstruction backbone provides a depth map and a camera pose for each frame. A lightweight predictor extrapolates the observed camera motion to any requested horizon h , and the last observed depth map is warped into that future viewpoint to serve as geometric conditioning. A flow-matching diffusion transformer, conditioned on the depth history, this warped depth map (the *anchor*), and h , then predicts the depth map at time $t+h$ directly, without recursive feedback. The horizon is thus an input to the model, and the anchor gives that input a physical meaning: it is the observation seen from where the camera is predicted to be at $t+h$.

a Explicit geometry at a requested future horizon

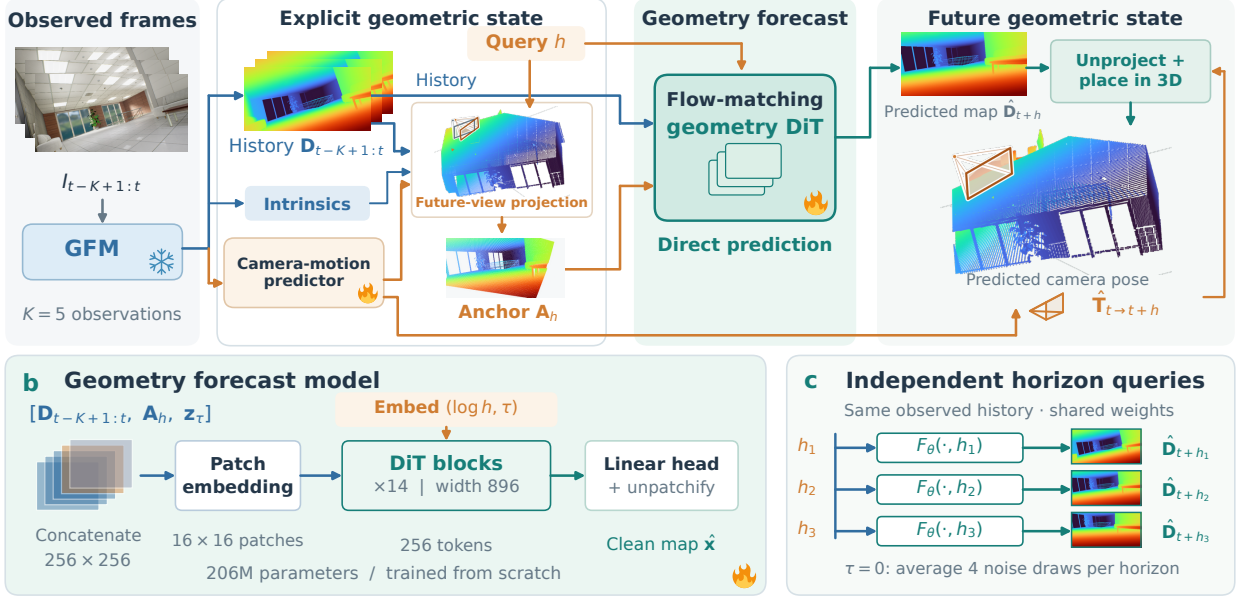


Fig. 2: Overview of GEOWM: (a) forecasting in explicit geometry, (b) the flow-matching transformer, and (c) independent horizon queries.

A. Problem Formulation

Let $I_{t-K+1:t}$ denote the K most recent frames. A frozen reconstruction backbone maps them to depth maps $\mathbf{D}_{t-K+1:t}$, camera intrinsics, and camera poses, from which we take the $K-1$ relative motions between consecutive frames. Given a positive horizon h from a dataset-defined set $\mathcal{H}$, the task is to predict the depth map $\mathbf{D}_{t+h}$ as seen from the camera at time $t+h$, and the pose $\mathbf{T}_{t \rightarrow t+h}$ of that camera relative to the last observed one, the transform that takes points from the camera frame at t into the one at $t+h$. We predict depth rather than point maps because, for a given camera, the pixel already fixes the viewing ray, so depth is the only free quantity per pixel; the point map follows from the predicted depth and pose. Every horizon is predicted from the same observed history, so no prediction at a shorter horizon is used as input for a longer one.

B. Camera-Motion Prediction and Geometric Conditioning

Camera-motion predictor. The $K-1$ observed relative motions are encoded as rotation vectors and normalised translation features. A lightweight predictor maps this history directly to the relative camera poses $\hat{\mathbf{T}}_{t \rightarrow t+h}$ of every future step in $\mathcal{H}$ at once. We use ridge regression for rotation and a small regressor for translation; the predictor has at most a few tens of thousands of parameters. Its accuracy is evaluated in Section IV against the camera forecasts of the baselines.

Geometric conditioning. Given $\hat{\mathbf{T}}_{t \rightarrow t+h}$ and the backbone’s intrinsics, we forward-project $\mathbf{D}_t$ into the predicted camera with nearest-depth visibility handling, an operation we write $\mathcal{W}$. The result,

$$\mathbf{A}_h = \mathcal{W}(\mathbf{D}_t, \hat{\mathbf{T}}_{t \rightarrow t+h}), \quad (1)$$

is the last observation seen from the predicted future viewpoint. The anchor supplies an estimate of the observed static structure in the future camera frame. We use it as conditioning only: the network predicts the complete future depth map, and it learns to account for errors in the observed depth and in the predicted motion, for scene dynamics, and for regions that become visible only in the future view.

C. Horizon-Conditioned Flow-Matching Transformer

Architecture. The network is a plain diffusion transformer [21] on depth maps. Its conditioning stack is $\mathbf{c}_h = (\mathbf{D}_{t-K+1:t}, \mathbf{A}_h)$, the observed depth maps and the anchor. These $K+1$ channels and the noisy target are stacked at 256×256 , split into 16×16 patches, and linearly projected to 256 tokens with a learned positional embedding. Fourteen pre-norm transformer blocks of width 896 follow, and a linear head with unpatchification gives the output. Spatial conditioning is by channel concatenation only; the horizon and the flow time enter through the normalisation layers, as described next. The flow time τ and $\log h$ are each sinusoidally embedded, concatenated, mapped by a small MLP, and injected into every block by adaptive layer normalisation with zero-initialised modulation (adaLN-zero [21]), so that the horizon modulates every layer. The transformer has 206M parameters and is trained from scratch; it operates directly on depth maps, with no learned autoencoder or tokenizer and no pretrained weight other than the frozen backbone.

Objective. We adopt a flow-matching formulation [22], [23] with clean-target prediction and a weighted L1 loss in velocity space. Let $\mathbf{x} = \mathbf{D}_{t+h}/\sigma_h$ be the normalised future-depth target, where σ_h is a fixed per-horizon normalisation constant computed on the training set. Let $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ be Gaussian noise and $\tau \in [0, 1]$ the flow time. The interpolant

is

$$\mathbf{z}_\tau = \tau \mathbf{x} + (1 - \tau) \epsilon, \quad (2)$$

so that $\tau = 0$ is pure noise and $\tau = 1$ is the target. The network predicts the clean target, $\hat{\mathbf{x}} = F_\theta(\mathbf{z}_\tau, \tau, h, \mathbf{c}_h)$ [24], [25], and the loss is applied to the velocity that $\hat{\mathbf{x}}$ implies. Writing x_p for the value of a map at pixel p , and taking the expectation over training windows, horizons, τ , and ϵ ,

$$\mathcal{L} = \mathbb{E} \left[\frac{1}{\sum_p w_p} \sum_p w_p \left| \frac{\hat{x}_p - z_{\tau,p}}{\max(1 - \tau, 0.05)} - (x_p - \epsilon_p) \right| \right]. \quad (3)$$

The per-pixel weight $w_p \propto m_p / \max(\mathbf{D}_{t+h,p}, d_{\min})$ is inverse depth over the valid-pixel mask m , bounded by a depth floor $d_{\min}$. It aligns the L1 objective with relative depth error, so that nearby structure is not swamped by the far field. A related term is used as an auxiliary loss in [25]; here it is the primary weight. Flow times follow a logit-normal distribution, $\tau = \text{sigmoid}(u)$ with $u \sim \mathcal{N}(-0.8, 0.8^2)$ [26]. Following [25], we set a fixed fraction of training samples to $\tau = 0$ to explicitly train the pure-noise state used at inference.

D. Training and Inference

Training. Each training example is an observed window together with a horizon h drawn uniformly from $\mathcal{H}$; the target is the ground-truth depth at $t + h$. The same network is therefore supervised at every horizon it will later be asked for, always from observed inputs. The transformer is trained from scratch on a single GPU in a few hours.

Inference. A single forward pass at $\tau = 0$ from pure noise returns a clean estimate, as in the one-step inference of [25]; we average $N = 4$ such passes with independent noise and invert the normalisation,

$$\hat{\mathbf{D}}_{t+h} = \frac{\sigma_h}{N} \sum_{n=1}^N F_\theta(\epsilon_n, 0, h, \mathbf{c}_h). \quad (4)$$

For the weighted L1 objective in Equation (3), evaluation at $\tau = 0$ reduces the training loss to the inverse-depth-weighted L1 error between $\hat{\mathbf{x}}$ and $\mathbf{x}$, so this read-out targets the relative-error criterion used by the metrics. Averaging over noise draws reduces the remaining dependence on ϵ , and no ODE integration is needed. Section IV compares this read-out with integrated samples. Each requested horizon needs a fixed number of network evaluations, independent of how far ahead it lies, and queries at different horizons can be batched; unlike a rollout, no intermediate future is computed or fed back. The predicted camera pose $\hat{\mathbf{T}}_{t \rightarrow t+h}$ is reported as is. For a static camera, as on DOMINO, the anchor coincides with $\mathbf{D}_t$ and is omitted, so the network is conditioned on the K depth maps alone.

IV. EXPERIMENTS

A. Datasets and Evaluation Protocol

Datasets. To show that GEOWM is robust across domains, we evaluate on four challenging datasets that span driving, aerial flight, and robotic manipulation. In every dataset, a

window consists of $K = 5$ observed frames, and the horizon set $\mathcal{H}$ is fixed per dataset. KITTI [27] provides 10 Hz driving sequences with LiDAR ground truth. We train on the 138 training drives of the KITTI depth benchmark and test on its 13 validation drives (3,257 windows), with stride 2 and 20 horizons from 0.2 s to 4.1 s. Cityscapes [28] provides 17 Hz stereo sequences. We train on the training split and evaluate on 500 validation windows, with stride 3 and five horizons from 0.18 s to 0.88 s. TartanAir [29] is a challenging aerial dataset of simulated flights through 18 photorealistic environments with fast 6-DoF camera motion. We hold out 37 of its 369 trajectories (7,314 windows) and predict 20 horizons of 1 to 20 frames. DOMINO [30] is one of the most challenging manipulation datasets. Episodes are observed from a static head camera, and objects move independently of the robot. It covers 35 tasks and five robot embodiments. Since the public release has RGB only, we re-render the stored trajectories in SAPIEN [31] to obtain ground-truth depth. To measure generalisation beyond the training distribution, we train on 23 tasks with one embodiment held out and test on four disjoint sets: seen tasks with seen robots (in-domain), novel tasks, a novel robot, and both novel (37,360 windows, 157,325 window-horizon pairs), with seven horizons from 0.24 s to 7.7 s.

Metrics. We evaluate three aspects of the forecast: the depth map, the camera motion, and the resulting 3D geometry. Depth is evaluated with the absolute relative error (AbsRel) at the target frame. Following VGGT-World [7], predictions are median-scaled per image on KITTI, Cityscapes, and TartanAir. Since manipulation requires depth at its true scale, on DOMINO we evaluate the raw output of our method. On KITTI, we average within each drive and then over the 13 drives. Camera motion is evaluated with the absolute trajectory error (ATE), relative translation error (RTE), and relative rotation error (RRE) after Umeyama alignment, as in VGGT-World. For 3D evaluation we follow the protocol of Gen3R [16]: predicted and ground-truth clouds are aligned with a similarity transform, voxel-downsampled at 5 mm and subsampled to 20k points each, and we report the mean of accuracy and completeness as the Chamfer distance (CD).

B. Implementation Details

While any feed-forward reconstruction model can serve as the backbone, we use a frozen VGGT-1B [2], which returns depth, intrinsics, and poses for the observed frames in one pass. Its depth is defined up to a scale per window, so during training the ground truth is brought to that scale with a factor computed from the last observed frame; on DOMINO, a single constant fitted on the training set converts the backbone’s depth to metres, so the network predicts depth at its true scale. Pixels without ground truth are masked, and ground-truth depth is capped at 80 m on the driving datasets and 100 m on TartanAir. The pure-noise fraction of training samples is 0.4 and the depth floor $d_{\min}$ of the inverse-depth weight is 0.05; on TartanAir, where scenes are mostly near-range, we omit this weight so as not to suppress the sparse far structure. We train with AdamW at a constant learning rate

TABLE I: **Future geometry forecasting on four datasets.** AbsRel ($\times 100$) and δ_1 (%) score the forecast depth, CD (m) the forecast 3D geometry after similarity alignment, and ATE (m), RTE (m), RRE (deg) the forecast camera trajectory. *Next* is the first predicted horizon; *Short*, *Mid*, and *Long* average the horizons given in each block, and *All* averages every horizon. Depth is median-scaled per image except on DOMINO, which is evaluated at true scale. **Best** and second best among copy-last, VGGT-World [7], Cosmos-3 [6], and GEOWM; copy-last repeats the last observed geometry.

Method	Next			Short			Mid			Long			All			Pose (trajectory-level)		
	AbsRel $\downarrow$	$\delta_1\uparrow$	CD $\downarrow$	AbsRel $\downarrow$	$\delta_1\uparrow$	CD $\downarrow$	AbsRel $\downarrow$	$\delta_1\uparrow$	CD $\downarrow$	AbsRel $\downarrow$	$\delta_1\uparrow$	CD $\downarrow$	AbsRel $\downarrow$	$\delta_1\uparrow$	CD $\downarrow$	ATE $\downarrow$	RTE $\downarrow$	RRE $\downarrow$
KITTI next 0.21 s, short 0.41–1.03 s, mid 1.24–2.07 s, long 2.27–4.14 s																		
Copy-last	9.1	90.1	0.754	13.9	82.6	0.900	18.9	75.7	1.152	22.5	71.8	1.406	19.2	75.8	1.209	6.82	1.33	0.83
DINO-Foresight	15.0	81.0	–	16.1	79.1	–	19.2	72.1	–	22.6	64.1	–	20.0	69.9	–	–	–	–
VGGT-World	7.8	92.4	0.737	11.2	87.8	0.796	21.4	73.9	1.263	29.1	57.6	2.123	22.5	69.5	1.574	3.43	1.39	1.50
Cosmos-3	<u>6.8</u>	<u>93.8</u>	<u>0.676</u>	<u>9.3</u>	<u>90.5</u>	<u>0.735</u>	<u>15.1</u>	83.5	0.957	<u>21.2</u>	<u>75.5</u>	<u>1.281</u>	<u>16.6</u>	<u>81.4</u>	<u>1.060</u>	<u>0.34</u>	<u>0.22</u>	0.67
GEOWM	6.6	94.3	0.539	8.9	90.6	0.654	13.3	<u>83.1</u>	0.957	17.4	76.5	1.259	14.2	81.9	1.027	0.23	0.10	0.67
CITYSCAPES next 0.18 s, short 0.35–0.53 s, mid 0.71–0.88 s																		
Copy-last	19.4	71.7	1.158	22.2	66.7	1.296	24.8	62.6	1.481	–	–	–	22.7	66.1	1.342	1.83	1.22	0.34
DINO-Foresight	20.1	68.9	–	21.1	67.1	–	22.6	64.2	–	–	–	–	21.5	66.3	–	–	–	–
VGGT-World	19.3	70.8	1.228	22.1	67.5	1.322	25.7	62.8	1.577	–	–	–	23.0	66.3	1.405	0.91	1.27	0.77
Cosmos-3	<u>17.7</u>	<u>74.6</u>	<u>1.119</u>	<u>18.7</u>	<u>72.6</u>	<u>1.158</u>	<u>20.1</u>	<u>70.6</u>	<u>1.258</u>	–	–	–	<u>19.1</u>	<u>72.2</u>	<u>1.190</u>	<u>0.08</u>	<u>0.10</u>	<u>0.31</u>
GEOWM	12.6	85.1	0.948	15.1	80.9	1.068	17.9	75.7	1.235	–	–	–	15.7	79.7	1.111	0.02	0.03	0.14
TARTANAIR next 1 frame, short 2–5, mid 6–10, long 11–20 frames																		
Copy-last	19.2	86.2	1.018	34.9	72.4	1.815	52.1	56.5	2.802	67.6	44.1	3.688	54.8	55.0	2.958	0.95	0.17	1.25
VGGT-World	19.9	84.6	1.137	38.2	68.4	2.088	54.0	46.3	3.980	<u>55.4</u>	36.6	4.683	49.8	47.8	3.811	0.80	0.34	1.99
Cosmos-3	<u>13.6</u>	<u>88.9</u>	<u>0.843</u>	<u>20.9</u>	<u>83.2</u>	<u>1.175</u>	<u>36.5</u>	<u>69.6</u>	<u>2.015</u>	<u>57.5</u>	<u>51.3</u>	<u>3.268</u>	<u>42.8</u>	<u>64.1</u>	<u>2.415</u>	<u>0.28</u>	<u>0.14</u>	<u>1.19</u>
GEOWM	12.0	89.8	0.786	18.6	84.5	1.094	28.9	73.3	1.844	40.1	57.3	3.151	31.6	68.3	2.294	0.13	0.09	0.77
DOMINO, IN-DOMAIN next 0.24 s, short 0.48–0.96 s, mid 1.92–2.88 s, long 4.80–7.68 s																		
Copy-last	<u>14.1</u>	<u>85.3</u>	<u>0.032</u>	<u>17.1</u>	<u>82.6</u>	<u>0.040</u>	21.6	<u>77.7</u>	<u>0.055</u>	27.5	<u>73.6</u>	<u>0.079</u>	20.9	<u>79.0</u>	<u>0.054</u>	–	–	–
VGGT-World	16.5	83.7	0.041	20.3	79.7	0.050	27.1	73.5	0.072	30.6	67.1	0.090	24.6	74.9	0.066	–	–	–
Cosmos-3	15.2	85.1	0.040	17.6	<u>82.6</u>	0.046	<u>20.8</u>	<u>78.4</u>	0.060	<u>25.8</u>	73.3	0.081	<u>20.5</u>	<u>79.1</u>	0.059	–	–	–
GEOWM	4.4	95.0	0.016	7.2	91.1	0.025	10.8	84.7	0.040	13.4	81.1	0.051	9.6	87.0	0.036	–	–	–

of 10^{-4} , weight decay 0.01, batch size 16, gradient clipping at 1.0, and bf16 precision for 100k steps, and report results with an exponential moving average of the weights (decay 0.999).

C. Main Results

Table I compares GEOWM with a recent, strong representative of each paradigm in Section II: Cosmos-3 [6], a state-of-the-art video world model trained on large-scale video data, a 16B model that we run zero-shot as released and read out with the same frozen backbone, and VGGT-World [7], a state-of-the-art latent world model, which forecasts in the latent space of that backbone and reaches long horizons by autoregressive rollout. For VGGT-World we use the released checkpoints on KITTI and Cityscapes, and train it ourselves with the authors’ model and recipe on TartanAir and DOMINO, for which no weights were released. DINO-Foresight [5], which forecasts self-supervised image features, is listed for reference on the two driving datasets, and copy-last marks what no forecasting at all achieves. GEOWM achieves the lowest AbsRel at every horizon group and the lowest all-horizon CD on all four datasets among the ranked methods. On all three moving-camera datasets, it also achieves the lowest ATE and RTE.

Depth forecasting. GEOWM has the lowest AbsRel at every horizon group on every dataset and the highest δ_1 in all but

one cell, and its margin grows with the horizon. On KITTI it matches Cosmos-3 at the first frame (6.6 vs. 6.8, within the seed-to-seed variation of this split) and leads by four points at the long horizons (17.4 vs. 21.2); on TartanAir the lead grows from 1.6 points at the first frame to 17 points at the long horizons (40.1 vs. 57.5). The rollout baseline shows the opposite trend: VGGT-World falls below copy-last from the mid horizons on KITTI and up to the mid horizons on TartanAir, which is the error accumulation that direct prediction avoids. On DOMINO, GEOWM cuts the error of every other method by about half at every horizon (9.6 vs. 20.5 for Cosmos-3 over all horizons).

3D geometry forecasting. The Chamfer distance follows the depth results: GEOWM has the lowest CD at every horizon group on all four datasets, with one tie against Cosmos-3 at the mid horizons of KITTI. Over all horizons, the margin over Cosmos-3 ranges from 3% on KITTI to 39% on DOMINO.

Camera-motion forecasting. On the three datasets with a moving camera, GEOWM has the lowest trajectory error on all three metrics, with a single tie against Cosmos-3 on the rotation error of KITTI. Its translation errors are clearly lower than those of Cosmos-3 (ATE 0.23 vs. 0.34 on KITTI, 0.02 vs. 0.08 on Cityscapes, 0.13 vs. 0.28 on TartanAir) and several times lower than those of VGGT-World, whose relative errors are worse than a static camera on all three

TABLE II: **Generalisation on DOMINO to unseen tasks and robots.** Same metrics, horizon groups, and ranking as the DOMINO block of Table I, which reports the seen tasks and robots.

Method	Next			Short			Mid			Long			All		
	AbsRel↓	δ_1 ↑	CD↓	AbsRel↓	δ_1 ↑	CD↓	AbsRel↓	δ_1 ↑	CD↓	AbsRel↓	δ_1 ↑	CD↓	AbsRel↓	δ_1 ↑	CD↓
DOMINO, NOVEL TASK															
Copy-last	14.1	85.4	0.032	16.0	83.1	0.039	22.6	77.6	0.055	30.9	71.6	0.075	21.9	78.6	0.053
VGGT-World	16.0	84.1	0.040	18.7	81.1	0.050	27.4	73.6	0.071	32.3	65.9	0.079	24.7	75.0	0.063
Cosmos-3	15.1	85.9	0.038	16.6	84.1	0.044	22.3	78.8	0.059	27.9	73.3	0.075	21.2	79.7	0.057
GEOWM	5.1	93.7	0.017	7.4	90.3	0.026	14.7	81.1	0.048	23.4	71.8	0.071	13.7	82.9	0.044
DOMINO, NOVEL EMBODIMENT															
Copy-last	11.3	87.6	0.030	13.4	84.4	0.036	16.1	80.4	0.047	20.9	75.5	0.072	16.0	81.2	0.049
VGGT-World	13.5	87.1	0.040	15.5	84.1	0.045	21.0	78.1	0.066	23.0	71.5	0.082	18.9	79.2	0.061
Cosmos-3	11.6	88.9	0.037	13.0	86.7	0.042	15.1	83.0	0.053	19.7	76.9	0.075	15.3	83.2	0.054
GEOWM	4.4	94.6	0.018	6.5	91.4	0.026	9.5	85.6	0.041	12.7	80.3	0.056	8.8	87.0	0.038
DOMINO, NOVEL TASK AND NOVEL EMBODIMENT															
Copy-last	10.9	88.4	0.029	12.8	85.4	0.036	15.7	80.4	0.050	21.1	72.4	0.082	15.7	80.7	0.052
VGGT-World	12.7	86.5	0.040	14.8	83.4	0.046	18.4	77.1	0.062	25.2	67.2	0.085	18.5	77.4	0.061
Cosmos-3	11.1	89.1	0.035	12.4	87.0	0.040	14.8	82.4	0.053	19.6	74.8	0.082	14.9	82.5	0.055
GEOWM	4.8	93.6	0.018	6.5	90.8	0.026	10.8	83.1	0.048	18.3	72.0	0.080	10.9	83.6	0.046

TABLE III: **Ablations on KITTI.** Variants of GEOWM, evaluated on the windows and with the protocol of Table I: median-scaled AbsRel ($\times 100$) and δ_1 (%), averaged over the KITTI horizon groups of that table, where *All* averages every horizon. N is the number of noise draws averaged at read-out.

Variant	Short		Long		All	
	AbsRel↓	δ_1 ↑	AbsRel↓	δ_1 ↑	AbsRel↓	δ_1 ↑
Copy-last	13.9	82.6	22.5	71.8	19.2	75.8
PREDICTION SPACE						
feature space	11.7	86.6	21.4	72.8	17.6	78.1
feature space, + depth loss	11.8	86.6	18.8	73.7	16.0	78.8
GEOMETRIC CONDITIONING						
no depth conditioning	29.1	43.9	28.0	47.3	28.6	45.5
uncompensated anchor	10.6	87.6	17.8	75.9	15.1	80.3
anchor, no history	9.4	89.9	18.7	74.5	15.1	80.5
last depth map only	12.8	84.9	18.3	75.3	16.2	78.8
FORECASTING STRATEGY						
one-step recursive rollout	11.3	87.7	38.9	53.0	27.7	66.5
two-step block rollout	9.7	90.0	31.4	60.6	22.4	72.5
OBSERVED FRAMES						
$K = 2$	10.1	89.2	18.7	75.1	15.4	80.4
$K = 3$	9.4	89.9	17.7	76.0	14.5	81.3
$K = 4$	9.3	90.2	17.6	76.3	14.4	81.6
READ-OUT						
Euler, 2 steps	9.8	90.1	18.1	76.2	14.9	81.4
Euler, 4 steps	11.6	88.0	19.7	74.0	16.6	79.2
Euler, 8 steps	14.0	84.2	22.4	70.1	19.2	75.4
GEOWM ($K=5$, anchor, $N=1$)	9.0	90.5	17.5	76.5	14.2	81.8
GEOWM ($K=5$, anchor, $N=4$)	8.9	90.6	17.4	76.5	14.2	81.9

datasets. This accuracy comes from a predictor with a few tens of thousands of parameters.

D. Generalisation to Novel Tasks and Robots

Table II tests what happens when the task, the robot, or both are unseen during training. GEOWM has the lowest AbsRel and the lowest Chamfer distance in every cell, and

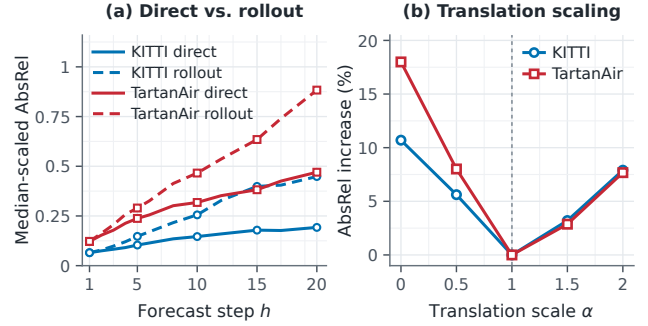


Fig. 3: **Direct forecasting and geometric conditioning.** (a) Direct prediction versus one-step recursive rollout with the same checkpoint. (b) Increase in horizon-averaged AbsRel when scaling the anchor's predicted translation relative to $\alpha = 1$; rotation is unchanged. Comparisons use matched window subsets and equal-drive averaging on KITTI.

it predicts the first horizon with an AbsRel between 4.4 and 5.1 where every other method is above 10. The rows differ in difficulty, so we read the results relative to copy-last: over all horizons, GEOWM reduces its error by 54% on seen tasks and robots, by 45% on an unseen robot, by 37% on unseen tasks, and by 31% when both are unseen. An unseen robot therefore costs little, whereas unseen tasks cost more. At the longest horizon of the two novel-task blocks Cosmos-3 has the higher δ_1 , and when both task and robot are unseen the AbsRel margin narrows to one point.

E. Ablation Studies

Table III varies the design choices of GEOWM on KITTI, with the evaluation windows and the protocol of Table I held fixed.

Forecasting in geometry space. To test the first pillar of the method we train a twin of GEOWM that keeps our architecture, our conditioning and our training schedule, but

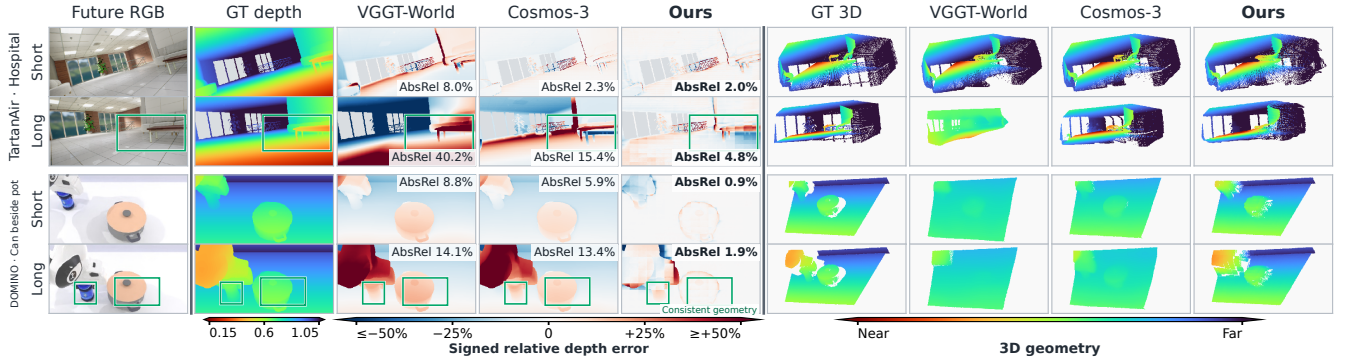


Fig. 4: **Qualitative forecasts.** Short/long horizons are $h = 1/10$ for TartanAir and $h = 4/48$ for DOMINO. Error maps show signed relative depth error (white is zero); boxes mark corresponding scene regions. Clouds use calibrated intrinsics, median-scaled TartanAir depths and fixed-scale DOMINO depths; colors indicate depth. All clouds share the same view without further alignment.

regresses the depth-head tokens of the frozen front end instead of depth, and decodes them back to depth with that same frozen head. Its targets are the four depth-head layers of that front end, reduced by a per-layer PCA to 128 standardised components each. The second variant is the stronger one: it adds a loss on the decoded depth, which aligns its objective with the evaluation metric and reduces its error over all horizons from 17.6 to 16.0, though not at the first frame. Forecasting in explicit geometry is better than both variants at every horizon group, by at least 2.8 points at the short horizons, 1.4 at the long ones and 1.8 over all horizons, and it uses half as many tokens per frame.

Geometric conditioning. Removing the depth conditioning altogether leaves an error worse than copy-last and almost unchanged by how far ahead the model is asked to predict. The anchor and the observed depth history matter at different ranges. Replacing the camera-compensated anchor by a plain copy of the last observed depth map costs most at the short horizons, whereas keeping the anchor and dropping the history costs most at the long ones. Using only the last observed depth map removes both, and is worse than removing either one alone at the short horizons and over all horizons. Fig. 3(b) asks whether the forecast uses the particular viewpoint the anchor encodes, or merely the presence of a warped depth map. Rebuilding the anchor with the predicted translation scaled by a factor α , rotation unchanged, traces a V whose minimum sits at $\alpha=1$, the predicted motion itself; removing the translation entirely costs 10.7% on KITTI and 18.0% on TartanAir. A model that ignored the viewpoint would trace a flat curve instead.

Direct prediction. The second pillar is tested by feeding the model its own predictions back into the context, with the same weights and the same windows. The first step is identical by construction, and the two strategies separate immediately afterwards: at the long horizons a one-step recursive rollout reaches 38.9 AbsRel against 17.4 for direct prediction, and falls behind copy-last. Predicting two steps from each context halves the number of feedback stages and recovers part of the gap without closing it. Fig. 3(a) shows the same behaviour over the full horizon range on two datasets, with rollout ending at 2.3 times the direct error

TABLE IV: **Inference cost.** Compute, parameters and the time to obtain the forecasts of different horizons.

Method	TFLOPs	Params		Time to forecast (s)			
		forecaster	backbone total	$h=1$	$h=5$	$h=10$	$h=20$
DINO-Foresight [5]	30.6	256 M	DINO 361 M	0.03	0.11	0.22	0.42
VGGT-World [7]	1116	433 M	VGGT 1.59 B	1.9	7.7	17.4	36.9
Cosmos-3 [6]	3362	17.0 B	VGGT 18.1 B	4.9	5.8	6.7	9.3
GEOWM	12.0	206 M	VGGT 1.36 B	0.11	0.11	0.11	0.11

on KITTI and 1.9 times on TartanAir.

Observed frames. Accuracy improves from two observed frames to four at every horizon group, and most of the gain is already reached at three. We use $K = 5$ in all other experiments.

Read-out. One-step prediction at $\tau = 0$ outperforms Euler integration at every horizon group, which supports the choice made in Section III-D. The degradation grows with the number of integration steps. Averaging four noise draws rather than one gives a small further gain at the short and long horizons, and leaves the all-horizon error unchanged at the precision reported here.

F. Qualitative Analysis

Fig. 4 shows forecasts on TartanAir and DOMINO at short and long horizons, as signed relative depth-error maps, white at zero error, and as the resulting point clouds. In the hospital scene, GEOWM preserves the room layout more faithfully as the viewpoint changes, while baseline errors grow around the furniture and structural boundaries. On DOMINO, it better retains the raised geometry of the can and pot, which the baselines tend to flatten, although object boundaries remain smooth.

G. Computational Cost

Table IV compares inference cost. A horizon is an input to GEOWM rather than a number of steps to take, so the same four-draw read-out answers any horizon and the latency is flat at 0.11 s across the evaluated range; producing all twenty horizons costs 12 TFLOPs. VGGT-World and DINO-Foresight instead extend their forecasts through

sequential rollout, so VGGT-World grows from 1.9 s to 36.9 s and spends 1116 TFLOPs over the same twenty horizons. Cosmos-3 does not roll out: it denoises a whole clip at once with a 17 B generator, which requires 4.9 s even at the nearest evaluated horizon. GEOWM therefore reaches the longest horizon over $300\times$ faster than VGGT-World and over $80\times$ faster than Cosmos-3, with the smallest forecaster of the four and the same frozen backbone as VGGT-World.

V. CONCLUSION

We presented GEOWM, a world model that forecasts directly in explicit geometry. GEOWM outperforms the evaluated video- and feature-based world models, while representation ablations further support predicting geometry directly. Direct horizon conditioning further avoids recursive feedback and its associated error accumulation, enabling efficient prediction at longer horizons. A lightweight camera-motion predictor improves the forecast by relating observed geometry to the future viewpoint. Across driving, aerial flight, and robotic manipulation, GEOWM produces accurate future geometry and camera-motion forecasts, while generalising to unseen manipulation tasks and robot embodiments. Together, these results establish GEOWM as an accurate and efficient approach to future geometry forecasting and demonstrate the value of combining explicit geometric prediction with direct horizon conditioning. Conditioning the forecast on robot actions is a natural follow-up. Additionally, predicting motion and geometry jointly is another direction for future work.

REFERENCES

- [1] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “DUST3R: Geometric 3D vision made easy,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 20 697–20 709.
- [2] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, “VGGT: Visual geometry grounded transformer,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 5294–5306.
- [3] Y. Wang, J. Zhou, H. Zhu, W. Chang, Y. Zhou, Z. Li, J. Chen, J. Pang, C. Shen, and T. He, “ π^3 : Permutation-equivariant visual geometry learning,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026.
- [4] N. V. Keetha, N. Müller, J. Schönberger, L. Porzi, Y. Zhang, T. Fischer, A. Knapitsch, D. Zauss, E. Weber, N. Antunes, J. Luiten, M. Lopez-Antequera, S. R. Bulò, C. Richardt, D. Ramanan, S. Scherer, and P. Kotschieder, “MapAnything: Universal feed-forward metric 3D reconstruction,” in *Proc. Int. Conf. 3D Vision (3DV)*, 2026, pp. 499–509.
- [5] E. Karypidis, I. Kakogeorgiou, S. Gidaris, and N. Komodakis, “DINO-Foresight: Looking into the future with DINO,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, pp. 163 779–163 811.
- [6] Aditi, N. Agarwal, A. Ali, J. Allen, M. Antolini, A. Aubame, A. Azzolini, J. Bai, M. Bala, Y. Balaji, J. Bapst *et al.*, “Cosmos 3: Omnimodal world models for physical AI,” *arXiv preprint arXiv:2606.02800*, 2026.
- [7] X. Sun, S. Wang, F. Zhang, L. Liu, C. Jia, Z. Song, Z. Huang, and Y. Luo, “VGGT-World: Transforming VGGT into an autoregressive geometry world model,” in *Proc. European Conf. Computer Vision (ECCV)*, 2026, pp. 382–400.
- [8] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 1171–1179.
- [9] B. Chen, D. M. Monsó, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann, “Diffusion forcing: Next-token prediction meets full-sequence diffusion,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 24 081–24 125.
- [10] X. Huang, Z. Li, G. He, M. Zhou, and E. Shechtman, “Self forcing: Bridging the train-test gap in autoregressive video diffusion,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [11] N. Agarwal, A. Ali, M. Bala *et al.*, “Cosmos world foundation model platform for physical AI,” *arXiv preprint arXiv:2501.03575*, 2025.
- [12] A. Wang, B. Ai, B. Wen *et al.*, “Wan: Open and advanced large-scale video generative models,” *arXiv preprint arXiv:2503.20314*, 2025.
- [13] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado, “GAIA-1: A generative world model for autonomous driving,” *arXiv preprint arXiv:2309.17080*, 2023.
- [14] S. Gao, J. Yang, L. Chen, K. Chitta, Y. Qiu, A. Geiger, J. Zhang, and H. Li, “Vista: A generalizable driving world model with high fidelity and versatile controllability,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 91 560–91 596.
- [15] H. Zhu, Y. Wang, J. Zhou, W. Chang, Y. Zhou, Z. Li, J. Chen, C. Shen, J. Pang, and T. He, “Aether: Geometric-aware unified world modeling,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, pp. 8535–8546.
- [16] J. Huang, Y. Yang, B. Yang, L. Ma, Y. Ma, and Y. Liao, “Gen3R: 3D scene generation meets feed-forward reconstruction,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 25 358–25 369.
- [17] Q. Zhang, S. Zhai, M. Á. B. Martin, K. Miao, A. Toshev, J. Susskind, and J. Gu, “World-consistent video diffusion with explicit 3D modeling,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 21 685–21 695.
- [18] H. Wu, D. Wu, T. He, J. Guo, Y. Ye, Y. Duan, and J. Bian, “Geometry forcing: Marrying video diffusion and 3D representation for consistent world modeling,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026.
- [19] F. Baldassarre, M. Szafraniec, B. Terver, V. Khalidov, F. Massa, Y. LeCun, P. Labatut, M. Seitzer, and P. Bojanowski, “Back to the features: DINO as a foundation for video world models,” *arXiv preprint arXiv:2507.19468*, 2025.
- [20] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zhulov, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas, “V-JEPA 2: Self-supervised video models enable understanding, prediction and planning,” *arXiv preprint arXiv:2506.09985*, 2025.
- [21] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 4195–4205.
- [22] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [23] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [24] T. Li and K. He, “Back to basics: Let denoising generative models denoise,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 36 115–36 125.
- [25] H. Xu, R. Wu, P. Henzler, N. Kalischek, M. Oechsle, F. Manhardt, M. Pollefeys, A. Geiger, F. Tombari, and M. Niemeyer, “PointDiT: Pixel-space diffusion for monocular geometry estimation,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2026.
- [26] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, and R. Rombach, “Scaling rectified flow transformers for high-resolution image synthesis,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2024, pp. 12 606–12 633.
- [27] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [28] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, “The Cityscapes dataset for semantic urban scene understanding,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3213–3223.
- [29] W. Wang, D. Zhu, X. Wang, Y. Hu, Y. Qiu, C. Wang, Y. Hu, A. Kapoor, and S. Scherer, “TartanAir: A dataset to push the limits of visual

- SLAM,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020, pp. 4909–4916.
- [30] H. Fang, S. Li, S. Wang, X. Xi, D. Liang, and X. Bai, “Towards generalizable robotic manipulation in dynamic environments,” in *Proc. European Conf. Computer Vision (ECCV)*, 2026.
- [31] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, L. Yi, A. X. Chang, L. J. Guibas, and H. Su, “SAPIEN: A SimulAted part-based interactive ENvironment,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 094–11 104.